Space Shuttle Main Engine

November 11, 2026

Contents

1 Introduction **1**

2 Oxygen **2**
2.1 First Enthalpy Exchanger 2
2.2 Combustion Chamber 3
2.3 Chamber Temperature (Second Enthalpy Exchanger) 7
2.4 Nozzle Performance 7

A Code listings **8**
A.1 SSMEMain.py 8
A.2 exchanger.py 10
A.3 Iterater.py 19

1 Introduction

The Space Shuttle Main Engine (SSME) is a liquid-fueled rocket engine used to power the Space Shuttle Orbiter. This report will discuss the design and methodologies of the SSME code, as well as the results of the code. For this project, we were given the parameters below:

Combustion Chamber Data	
Fuel	Hydrogen
Oxidizer	O_2
Fuel/Oxidizer Ratio	0.166
Hydrogen Injection Temperature	850 K
Oxidizer Injection Temperature	530 K
Chamber Pressure	204 atm

R_t	5.15 inch
R_e	45.35 inch
θ_p	32 degree

2 Oxygen

2.1 First Enthalpy Exchanger

We begin with computing the equivalence ratio ϕ for the oxygen first enthalpy exchanger. The equivalence ratio is defined as the ratio of the mass of fuel to the mass of oxidizer. The mass of fuel is the mass of hydrogen, and the mass of oxidizer is the mass of oxygen. The mass of hydrogen is $\phi \frac{mol_{O_2}}{mol_{H_2}} = \Phi$.

From here we are to calculate the stndard heat of formation by:

$$\Delta\hat{h}_{H_2} = \hat{h}_{H_2}(850) + \hat{R}T^2 \int_{pf}^{pi} \frac{1}{p} \frac{-\hat{b}_{H_2} p}{\hat{R}T^2} dp \tag{1}$$

$$\Delta\hat{h}_{O_2} = \hat{h}_{O_2}(530) + \hat{R}T^2 \int_{pf}^{pi} \frac{1}{p} \frac{-\hat{b}_{O_2} p}{\hat{R}T^2} dp \tag{2}$$

Species	$\hat{b}$
H_2	0.0266
O_2	0.0318
H_2O	0.03049
N_2	0.0391
F_2	0.02896
HF	0.0739

Where $\hat{b}$ is read from a table developed in `exchanger.py` file. $\Delta\hat{h}_i$ is calculated from both hydrogen and oxygen. Once we calculate $\Delta\hat{h}_i$ for each, we can calculate the enthalpy of the mixture.

$$\Delta\hat{h}_{mix} = \Phi\Delta\hat{h}_{H_2} + \Delta\hat{h}_{O_2} \tag{3}$$

With $\Phi = 2\phi$ moles of H_2 per mole of O_2, complete combustion (no dissociation) gives

$$O_2 + \Phi H_2 \rightarrow 2H_2O + (\Phi - 2)H_2 \tag{4}$$

The energy available to heat the products is the propellant enthalpy plus the heat released forming two moles of water:

$$\Delta\hat{h}_{total} = \Phi\Delta\hat{h}_{H_2} + \Delta\hat{h}_{O_2} - 2\Delta\hat{h}^{298}_{f,H_2O} \tag{5}$$

The first estimate of the chamber temperature, T_{FEE}, is the single temperature at which the products absorb exactly that energy. It is found with `scipy.optimize.brentq`:

$$2\left[\hat{h}_{H_2O}(T_{FEE}) - \hat{h}_{H_2O}(298)\right] + (\Phi - 2)\left[\hat{h}_{H_2}(T_{FEE}) - \hat{h}_{H_2}(298)\right] = \Delta\hat{h}_{total} \tag{6}$$

This gives $T_{FEE} \approx 4617$ K. Because it ignores dissociation it is an upper bound; the chamber temperature comes from the second enthalpy exchanger in Section 2.3.

2.2 Combustion Chamber

For the Combustion Chamber, we have the two equations below:

$$H_2O \rightarrow OH + H \tag{7}$$

$$H_2 \rightarrow 2H \tag{8}$$

The Gibbs Free Energy for each species is given by:

$$\hat{g}_{H_2O} = \texttt{h2o_table_g}(T_C) \tag{9}$$

$$\hat{g}_{H_2} = \texttt{h2_table_g}(T_C) \tag{10}$$

$$\hat{g}_{OH} = \texttt{oh_table_g}(T_C) \tag{11}$$

$$\hat{g}_{H} = \texttt{h_table_g}(T_C) \tag{12}$$

We can then calculate the Gibbs Free Energy of the reaction by:

$$\Delta\hat{g}_{H_2} = 2\hat{g}_H - \hat{g}_{H_2} \tag{13}$$

$$\Delta\hat{g}_{H_2O} = \hat{g}_H + \hat{g}_{OH} - \hat{g}_{H_2O} \tag{14}$$

Now is the point where we involve the **Law of Mass Action**:

Species	$\nu'_{(i)}$	$\nu''_{(i)}$	$\nu''_{(i)} - \nu'_{(i)}$
H_2O	1	0	-1
OH	0	1	1
H	0	1	1

We an now use:

$$K_n = \prod_{i=1}^{N_s} X_{(n)}^{(\nu''_{(i)} - \nu'_{(i)})} = (\frac{p}{p_a})^{-\sigma_v} K_p \tag{15}$$

Where;

$$\sigma_v = \sum_{i=1}^{N_s} (\nu''_{(i)} - \nu'_{(i)}) \tag{16}$$

$$K_p = e^{-\frac{\Delta G}{\hat{R}T}} \tag{17}$$

$$\Delta G = \sum_{i=1}^{N_s} (\nu''_{(i)} - \nu'_{(i)})\hat{g}_{(i)} \tag{18}$$

Both reactions have $\sigma_v = +1$, so we can solve for the equilibrium constant K_n.

$$K_{H_2O} = \left(\frac{204atm}{1atm}\right)^{-1} e^{-\frac{\Delta\hat{g}_{H_2O}}{\hat{R}T_C}} \tag{19}$$

$$K_{H_2} = \left(\frac{204atm}{1atm}\right)^{-1} e^{-\frac{\Delta\hat{g}_{H_2}}{\hat{R}T_C}} \tag{20}$$

Now we move into the chemistry problem.
Atom Balance:

$$H_2 + lO_2 \rightarrow N_{H_2O}H_2O + N_{H_2}H_2 + N_H H + N_{OH}OH \tag{21}$$

Recall, due to the fule being Hydrogen;

$$mass_{fuel} = 2.02 \tag{22}$$

$$mass_{Oxidizer} = \frac{mass_{fuel}}{\phi} \tag{23}$$

$$l = \frac{mass_{Oxidizer}}{2\hat{m}_{O_2}} \tag{24}$$

Where $\hat{m}_{O_2}$ is the molar mass of Oxygen. And ϕ is the Fuel to Oxidizer ratio, and not the equivalence ratio. Revisiting the **Atom Balance** equation, we can now solve for the number of moles of each species.

The atom balances for hydrogen and oxygen are:

$$H: \quad 2 = 2N_{H_2O} + 2N_{H_2} + N_H + N_{OH} \tag{25}$$

$$O: \quad 2l = N_{H_2O} + N_{OH} \tag{26}$$

That is four unknowns with only two equations, so the Law of Mass Action supplies the other two. The mole fractions are normalized by the full total,

$$X_{(i)} = \frac{N_{(i)}}{N_{Total}}, \qquad N_{Total} = N_{H_2O} + N_{H_2} + N_H + N_{OH}, \tag{27}$$

so the two equilibrium relations become

$$K_{H_2O} = \frac{X_H X_{OH}}{X_{H_2O}} = \frac{N_H N_{OH}}{N_{H_2O} N_{Total}} \tag{28}$$

$$K_{H_2} = \frac{X_H^2}{X_{H_2}} = \frac{N_H^2}{N_{H_2} N_{Total}} \tag{29}$$

The four coupled equations (two atom balances and two equilibrium relations) are solved with `scipy.optimize.fsolve` in terms of $\ln N_{(i)}$, which keeps every mole number positive. The starting guess is complete combustion, $N_{H_2O} = 2l$ and $N_{H_2} = 1 - 2l$. At the chamber temperature T_c (Section 2.3) the solution is:

$$N_{H_2O} = 0.6763, \quad N_{H_2} = 0.2559, \quad N_H = 0.0530, \quad N_{OH} = 0.0827 \tag{30}$$

Species	$N_{(i)}$	$X_{(i)}$	$m_{(i)}$	$Y_{(i)}$	$\hat{c}_{p(i)}$	$c_{p(i)}$	$Y_{(i)}c_p$
H_2O	N_{H_2O}	$X_{H_2O} = \frac{N_{H_2O}}{N_{Total}}$	$\hat{m}N_{H_2O}$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(H_2O)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	$Y_{H_2O}c_p$
H_2	N_{H_2}	$X_{H_2} = \frac{N_{H_2}}{N_{Total}}$	$\hat{m}N_{H_2}$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(H_2)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	$Y_{H_2}c_p$
H	N_H	$X_H = \frac{N_H}{N_{Total}}$	$\hat{m}N_H$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(H)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	Y_Hc_p
OH	N_{OH}	$X_{OH} = \frac{N_{OH}}{N_{Total}}$	$\hat{m}N_{OH}$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(OH)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	$Y_{OH}c_p$

Now we can calculate c_p, we sum the $Y_{(i)}c_p$ values for each species.

$$c_p = \sum_i Y_{(i)}c_p \tag{31}$$

$$c_p = Y_{H_2O}c_p + Y_{H_2}c_p + Y_Hc_p + Y_{OH}c_p \tag{32}$$

We perform now calulate the gas constant R.

$$R = \frac{\hat{R}}{m_{Total}} \tag{33}$$

where $m_{Total} = \sum_i X_{(i)}m_{(i)}$ is the mixture molar mass. The mole fractions are normalized by $N_{Total} = N_{H_2O} + N_{H_2} + N_H + N_{OH}$, which exceeds one once dissociation produces H and OH.

$$c_v = c_p - R \tag{34}$$

$$\gamma = \frac{c_p}{c_p - \frac{\hat{R}}{\sum_i X_{(i)}m_{(i)}}} = \frac{c_p}{c_p - \frac{\hat{R}}{m_{Total}}} = \frac{c_p}{c_p - R} = \frac{c_p}{c_v} \tag{35}$$

2.3 Chamber Temperature (Second Enthalpy Exchanger)

Breaking H_2O and H_2 apart into H and OH absorbs energy, so the dissociated products reach a lower temperature than T_{FEE}. The composition depends on temperature through K_n, and the temperature depends on the composition through the energy balance, so the two are solved together. For a trial temperature the combustion-chamber step gives the equilibrium mixture, and the energy balance per mole of O_2 is checked:

$$\sum_i N_{(i)} \left[\Delta\hat{h}^{298}_{f(i)} + \hat{h}_{(i)}(T_c) - \hat{h}_{(i)}(298) \right] = \Phi \Delta\hat{h}_{H_2} + \Delta\hat{h}_{O_2} \tag{36}$$

`brentq` adjusts the temperature between 2000 K and T_{FEE} until both sides match. The sensible enthalpies of H_2, O_2 and H_2O come straight from the tables, H uses its constant c_p, and OH is built from its tabulated enthalpy of formation:

$$\hat{h}_{OH}(T) - \hat{h}_{OH}(298) = \Delta\hat{h}_{f,OH}(T) - \Delta\hat{h}_{f,OH}(298) + \frac{1}{2}\left[\hat{h}_{H_2}(T) - \hat{h}_{H_2}(298)\right] + \frac{1}{2}\left[\hat{h}_{O_2}(T) - \hat{h}_{O_2}(298)\right] \tag{37}$$

The same construction applied to H_2O reproduces its tabulated sensible enthalpy within 0.07%. The result is $T_c \approx 3949$ K, which is used for γ, R and the nozzle. A Cantera equilibrium calculation with the same inputs gives 3881 K; most of the difference is that this model leaves out O and O_2.

2.4 Nozzle Performance

We begin by defining the Area Ratio:

$$A_r = \frac{\pi R_e^2}{\pi R_t^2} \tag{38}$$

Allowing for us to solve for the Mach number:

$$A_r = \left(\frac{\gamma+1}{2}\right)^{-\frac{\gamma+1}{2(\gamma-1)}} \frac{1}{M_e}\left[1 + \frac{\gamma-1}{2}M_e^2\right]^{\frac{\gamma+1}{2(\gamma-1)}} \tag{39}$$

We can solve for Mach number M_e by making use of the `scipy.optimize.fsolve` function, which returns the roots of a non-linear function. After solving for M_e we can calculate the exit velocity v_e, exit Pressure p_e and exit Temperature T_e.

$$T_e = T_c \left[1 + \frac{\gamma - 1}{2} M_e^2\right]^{-1} \tag{40}$$

$$p_e = p_c \left[1 + \frac{\gamma - 1}{2} M_e^2\right]^{-\frac{\gamma}{\gamma - 1}} \tag{41}$$

$$v_e = M_e \sqrt{\gamma R T_e} \tag{42}$$

After solving for (Eq. 42), (Eq. 41) and (Eq. 40) we can calculate the mass flow rate $\dot{m}$.

$$\dot{m} = \frac{A_t p_c \sqrt{\gamma}}{\sqrt{R T_c}} \left[\frac{2}{\gamma + 1}\right]^{\frac{\gamma+1}{2(\gamma-1)}} \tag{43}$$

Then once you have the mass flow rate you can calculate the thrust $\mathcal{T}$, specific impulse I_{sp} and the thrust coefficient C_T.

$$\mathcal{T} = \dot{m} v_e + (p_e - p_a) A_e \tag{44}$$

where p_a is the ambient pressure ($p_a = 0$ in vacuum, $p_a = 101325$ Pa at sea level).

$$I_{sp} = \frac{\mathcal{T}}{\dot{m} g_0} \tag{45}$$

$$C_T = \frac{\mathcal{T}}{p_c A_t} \tag{46}$$

A Code listings

A.1 SSMEMain.py

```
import pandas as pd
from exchanger import *


if __name__ == '__main__':

    p_starting = 204 #starting pressure in atm
    p_starting = p_starting * 101325 #convert to pascals
```

```
    FuelOxidizer_ratio = 0.166
    # equivalence ratio: (mol H2 per mol O2) / 2, stoichiometric being 2 H2 + O2
    phi = (FuelOxidizer_ratio * MOLAR_MASS['O2'] / MOLAR_MASS['H2'])/2
    Hydr_inject_temp = 850 #K
    Ox_inject_temp = 530   #K
    ThroatRadius = 5.15 #inches
    ExitRadius = 45.35 #inches
    FEE_T, FEE_del_h_total, del_h2_hat, b_hat_table, h_reactants = FEE(Hydr_inject_temp, p_starting, Ox_inject_temp, phi)
    print('phi = ', phi)

    print('Temperature of the first enthalpy exchanger (complete combustion) = ', FEE_T, 'K')
    print('Total enthalpy change in the first enthalpy exchanger = ', FEE_del_h_total, 'J')

    # second enthalpy exchanger: equilibrium (dissociated) products carry the reactant enthalpy
    T_c = ChamberTemperature(FEE_T, p_starting, FuelOxidizer_ratio, h_reactants)
    print('Chamber temperature (equilibrium, with dissociation) = ', T_c, 'K')

    gamma, R, cp_total, cv, LawOfMassAction_df = OCC(T_c, p_starting, FuelOxidizer_ratio)
    print('gamma = ', gamma)

    p_sea_level = 101325 # Pa
    m_dotOx, v_eOx, p_eOx, ThrustOx, c_TOx, I_spOx, T_e_Ox, M_eOx = OxNozzlePerf(ThroatRadius, ExitRadius, gamma, T_c, R, p_starting)
    _, _, _, ThrustOx_SL, c_TOx_SL, I_spOx_SL, _, _ = OxNozzlePerf(ThroatRadius, ExitRadius, gamma, T_c, R, p_starting, p_sea_level)

    print('Ox exit Mach number = ', M_eOx)
    print('Ox mass flow rate = ', m_dotOx, 'kg/s')
    print('Ox exit velocity = ', v_eOx, 'm/s')
    print('Ox exit pressure = ', p_eOx, 'Pa')
    print('Ox exit temperature = ', T_e_Ox, 'K')
    print('Ox thrust (vacuum) = ', ThrustOx, 'N')
    print('Ox thrust coefficient (vacuum) = ', c_TOx)
```

```
    print('Ox specific impulse (vacuum) = ', I_spOx, 's')
    print('Ox thrust (sea level) = ', ThrustOx_SL, 'N')
    print('Ox thrust coefficient (sea level) = ', c_TOx_SL)
    print('Ox specific impulse (sea level) = ', I_spOx_SL, 's')


    print('Flourine Problem')
    b_hat_table, FEE_T_flourine = FEE_F(phi, p_starting, del_h2_hat, b_hat_table, Ox_inject_temp)

    print('Temperature of the first enthalpy exchanger (Flourine) = ', FEE_T_flourine, 'K')

    FCC(FEE_T_flourine)
```

A.2 exchanger.py

```
from Iterater import *
from scipy.optimize import fsolve
import pandas as pd
import numpy as np
from scipy.optimize import brentq

# Molar masses in g/mol, used everywhere (SSMEMain, FEE, OCC)
MOLAR_MASS = {'H2': 2.01588, 'O2': 31.9988, 'H2O': 18.01528, 'OH': 17.00734, 'H': 1.00794}
R_HAT = 8.314  # J/mol-K


def FEE(Hydr_inject_temp, p_starting, Ox_inject_temp, phi):
    '''
    This is a function that returns the calculations of the first enthalpy exchanger
    :param Hydr_inject_temp: temperature of hydrogen injection in K
    :param p_starting: starting pressure in Pa
    :param Ox_inject_temp: temperature of oxygen injection in K
    :param phi: equivalence ratio
    :return T the complete-combustion (no dissociation) temperature in K: the first estimate
    :return del_h_total enthalpy available to heat the products, J per mol O2
    :return del_h2_hat the enthalpy of the injected H2, J/mol
```

```
    :return b_hat_table the table of b_hat values
    :return h_reactants enthalpy of the injected propellants, J per mol O2
    '''


    # Van der Waals co-volume b_hat (L/mol)
    b_hat_table = pd.DataFrame({'substance': ['H2', 'H2O', 'O2', 'N2'],
                                'b_hat': [0.0266, 0.03049, 0.0318, 0.0391]})

    b_hat_h2 = b_hat_table['b_hat'][0]
    b_hat_o2 = b_hat_table['b_hat'][2]

    # injected propellant enthalpy (J/mol): sensible + real-gas pressure correction b*(p - p_ref)
    del_h2_hat = h2_tableH(Hydr_inject_temp) + b_hat_h2*(p_starting - 101325)/1000
    del_o2_hat = o2_tableH(Ox_inject_temp) + b_hat_o2*(p_starting - 101325)/1000
    mols_h2 = phi*2
    mols_o2 = 1

    # Complete combustion, per mole of O2: O2 + 2phi H2 -> 2 H2O + (2phi - 2) H2
    # Energy available to heat the products = reactant enthalpy + heat released forming 2 H2O
    del_h_total = del_h2_hat*mols_h2 + del_o2_hat*mols_o2 - 2*H_F_298['H2O']

    # First estimate of the chamber temperature: the single T at which the products'
    # sensible enthalpy equals that energy (no dissociation yet, so this is an upper bound)
    def energy_residual(T):
        return 2*sensible_enthalpy('H2O', T) + (mols_h2 - 2)*sensible_enthalpy('H2', T) - del_h_total

    T = brentq(energy_residual, 300, 4900)
    h_reactants = del_h2_hat*mols_h2 + del_o2_hat*mols_o2

    return T, del_h_total, del_h2_hat, b_hat_table, h_reactants

```

```
54
55 def ChamberTemperature(FEE_T, p_starting, FO_ratio,
       h_reactants):
56     '''
57     Adiabatic chamber temperature with dissociation (the
       second enthalpy exchanger).
58     Finds the T at which the equilibrium products from OCC
       carry the same enthalpy
59     as the injected propellants. Dissociation absorbs energy,
       so the answer lies
60     below the complete-combustion estimate FEE_T.
61     :param FEE_T: complete-combustion temperature from FEE in
       K (upper bracket)
62     :param p_starting: chamber pressure in Pa
63     :param FO_ratio: fuel to oxidizer mass ratio
64     :param h_reactants: enthalpy of the injected propellants,
       J per mol O2
65     :return T_c: chamber temperature in K
66     '''
67     l = MOLAR_MASS['H2']/(FO_ratio*MOLAR_MASS['O2'])  # mol
       O2 per mol H2, as in OCC
68     species = ['H2O', 'H2', 'H', 'OH']                  #
       order of OCC's mole table
69
70     def energy_residual(T):
71         mols = OCC(T, p_starting, FO_ratio)[4]['mols'][0:4]
72         # OCC works per mol H2; dividing by l puts the
       products per mol O2
73         h_products = sum(float(n)/l*total_enthalpy(sp, T) for
        sp, n in zip(species, mols))
74         return h_products - h_reactants
75
76     return brentq(energy_residual, 2000, FEE_T, xtol=1e-6)
77
78 def OCC(FEE_T, p_starting, FO_ratio):
79     '''
80     This is a function that returns the calculations of the
       oxygen combustion chamber
81     :param FEE_T: temperature of the first enthalpy exchanger
       in K
82     :param p_starting: starting pressure in Pa
83     :param FO_ratio: fuel to oxidizer ratio
84     :return gamma: the ratio of specific heats
85     :return R: the gas constant
86     :return cp_total: the total specific heat capacity
```

```
    :return cv : the specific heat capacity at constant volume
    :return LawOfMassAction_df: the dataframe of the law of mass action
    '''

    # The chemical equation is H2O -> H + OH
    molfrac_h2o_react = 1
    molfrac_h_react = 0
    molfrac_oh_react = 0
    molfrac_h2o_prod = 0
    molfrac_h_prod = 1
    molfrac_oh_prod = 1

    # The chemical equation is H2 -> 2H
    molfrac_h2_react = 1
    molfrac_2h_react = 0
    molfrac_2h_prod = 2
    molfrac_h2_prod = 0

    molfrac_h2o_total = molfrac_h2o_prod - molfrac_h2o_react
    molfrac_h_total = molfrac_h_prod - molfrac_h_react
    molfrac_oh_total = molfrac_oh_prod - molfrac_oh_react
    sigma_nu = molfrac_h2o_total + molfrac_h_total + molfrac_oh_total
    # print('sigma_nu = ', sigma_nu)

    molfrac_h2_total = molfrac_h2_prod - molfrac_h2_react
    molfrac_2h_total = molfrac_2h_prod - molfrac_2h_react
    sigma_nu = molfrac_h2_total + molfrac_2h_total
    # print('sigma_nu = ', sigma_nu)

    # Gibbs free energy of formation
    g_f_h2o = h2o_table_g(FEE_T)
    g_f_h = h_table_g(FEE_T)
    g_f_oh = oh_table_g(FEE_T)
    g_f_h2 = h2_table_g(FEE_T)

    # Gibbs free energy of reaction: dG = sum_i (nu''_(i) - nu'_(i)) g_f(i), i.e. products minus reactants
    g_r_h2 = 2*g_f_h - g_f_h2             # H2  -> 2H
    g_r_h2o = g_f_h + g_f_oh - g_f_h2o    # H2O -> H + OH

    k_h2 = ((p_starting/101325)**-1)*np.exp(-g_r_h2/(R_HAT*FEE_T))
```

```
127     k_h2o = ((p_starting/101325)**-1)*np.exp(-g_r_h2o/(R_HAT*
    FEE_T))
128 
129 
130     # k_h2o = 0.007149
131     # k_h2 = 0.011874839
132 
133     # mol O2 per mol H2 fed: (1/FO_ratio) kg O2 per kg H2,
    converted to moles
134     l = MOLAR_MASS['H2']/(FO_ratio*MOLAR_MASS['O2'])
135 
136     # Products per mole of H2 fed: H2 + l O2 -> N_H2O H2O +
    N_H2 H2 + N_H H + N_OH OH
137     #   H atoms:  2N_H2O + 2N_H2 + N_H + N_OH = 2
138     #   O atoms:   N_H2O + N_OH               = 2l
139     #   H2O -> H + OH:  X_H X_OH / X_H2O = N_H N_OH / (N_H2O
    N_total) = k_h2o
140     #   H2  -> 2H:      X_H^2 / X_H2     = N_H^2   / (N_H2
    N_total)   = k_h2
141     # Solved in log(N) so every mole number stays positive (
    no root picking needed).
142     def residuals(log_n):
143         n_h2o, n_h2, n_h, n_oh = np.exp(log_n)
144         n_tot = n_h2o + n_h2 + n_h + n_oh
145         return [2*n_h2o + 2*n_h2 + n_h + n_oh - 2,
146                 n_h2o + n_oh - 2*l,
147                 np.log(n_h*n_oh/(n_h2o*n_tot)) - np.log(k_h2o
    ),
148                 np.log(n_h**2/(n_h2*n_tot)) - np.log(k_h2)]
149 
150     # start from complete combustion with a little
    dissociation
151     guess = np.log([2*l, 1 - 2*l, 0.01, 0.01])
152     log_n, info, ier, msg = fsolve(residuals, guess,
    full_output=True, xtol=1e-12)
153     if ier != 1 or max(abs(r) for r in residuals(log_n)) > 1e
    -9:
154         raise RuntimeError(f'OCC equilibrium solve did not
    converge: {msg}')
155 
156     N_h2o, N_h2, N_h, N_oh = (float(n) for n in np.exp(log_n)
    )
157     N_total = N_h + N_oh + N_h2 + N_h2o
158 
159     molar_mass_h2o = MOLAR_MASS['H2O']
```

```
    molar_mass_h = MOLAR_MASS['H']
    molar_mass_oh = MOLAR_MASS['OH']
    molar_mass_h2 = MOLAR_MASS['H2']

    LawOfMassAction_df = pd.DataFrame({'substance': ['N_H2O',
    'N_H2', 'N_H', 'N_OH', 'Total'],
                                       'mols': [N_h2o, N_h2,
    N_h, N_oh, N_total]})
    # Mole fractions must be normalized by the total moles:
    once H and OH form,
    # N_total = N_H2O + N_H2 + N_H + N_OH > 1, so X_(i) != N_
    (i).
    LawOfMassAction_df['X_(i)'] = LawOfMassAction_df['mols']
    / N_total
    # X_(i)*m_hat_(i): the species' contribution to the
    mixture molar mass (g/mol)
    LawOfMassAction_df['m_hat_(i)'] = pd.Series([
    molar_mass_h2o, molar_mass_h2, molar_mass_h, molar_mass_oh
    , 0.0])\
                                        * LawOfMassAction_df['
    X_(i)']

    # mixture molar mass m_hat = sum_i X_(i) m_hat_(i), over
    the four species
    m_hat_total = sum(LawOfMassAction_df['m_hat_(i)'][0:4])
    LawOfMassAction_df.loc[4, 'm_hat_(i)'] = m_hat_total
    # mass fractions Y_(i) = X_(i) m_hat_(i) / m_hat
    LawOfMassAction_df['Y_(i)'] = LawOfMassAction_df['m_hat_(
    i)'] / m_hat_total

    h2_cp = h2_table_cp(FEE_T)
    h2o_cp = h2o_table_cp(FEE_T)
    h_cp = h_table_cp(FEE_T)
    oh_cp = oh_table_cp(FEE_T)

    cp_hats = pd.Series([h2o_cp, h2_cp, h_cp, oh_cp])
    cps = cp_hats / pd.Series([molar_mass_h2o, molar_mass_h2,
    molar_mass_h, molar_mass_oh])
    cp_df = pd.DataFrame({'cp_hat': cp_hats, 'cp': cps})
    cp_df['Y_(i)*cp_(i)'] = cp_df['cp'] * LawOfMassAction_df[
    'Y_(i)'][0:4]

    cp_total = sum(cp_df['Y_(i)*cp_(i)'])

    LawOfMassAction_df = pd.concat([LawOfMassAction_df, cp_df
```

```
    ], axis=1)
192
193     R = R_HAT/m_hat_total
194     cv = cp_total - R
195
196     gamma = cp_total/cv
197
198     return gamma, R, cp_total, cv, LawOfMassAction_df
199
200 def FEE_F(phi, p_starting, del_h2_hat, b_hat_table,
    Ox_inject_temp):
201     '''
202     This is a function that returns the calculations of the
    fuel enthalpy exchanger - flourine
203     :param phi the equivalence ratio
204     :param p_starting combustion pressure in Pa
205     :param del_h2_hat heat of formation of H2 from FEE -
    Oxygen
206     :param b_hat_table Van der Waals table of constants
207     :param Ox_inject_temp: Oxidizer injection temperature in
    K
208     :return T the temperature of the FEE in K
209     :return b_hat_table the Van der Waals table of constants
    - edited
210     '''
211
212     b_hat_table = pd.concat([b_hat_table,
213                              pd.DataFrame({'substance': ['F2'
    , 'HF'], 'b_hat': [0.02896, 0.0739]})],
214                             ignore_index=True)
215
216     b_hat_f2 = b_hat_table['b_hat'][3]
217     F2_inject_temp = Ox_inject_temp
218
219     del_f2_hat = f2_tableH(F2_inject_temp) + b_hat_f2*(
    p_starting - 101325)/1000
220     del_hf_hat = abs(hf_tableH(298))
221     del_h_total = phi*del_h2_hat + del_f2_hat
222     del_h_total = del_h_total + del_hf_hat
223
224
225     r = phi - 1
226     q = 1
227
228     r_perc = r/(r+q)
```

```
229     q_perc = 1-r_perc
230 
231     T_h2 = h2_tableT(del_h_total)
232     T_hf = hf_tableT(q_perc*del_h_total)
233 
234     T = (T_hf + T_h2) / 2
235 
236 
237     return b_hat_table, T
238 
239 def OxNozzlePerf(ThroatRadius, ExitRadius, gamma, FEE_T, R,
    p_starting, p_ambient=0.0):
240     '''
241     This is a function that returns the performance of the
    oxidizer nozzle
242     :param ThroatRadius: throat radius in inches
243     :param ExitRadius: exit radius in inches
244     :param gamma: ratio of specific heats (frozen, from OCC)
245     :param FEE_T: chamber temperature in K
246     :param R: gas constant in kJ/kg-K
247     :param p_starting: chamber pressure in Pa
248     :param p_ambient: ambient pressure in Pa (0 = vacuum,
    101325 = sea level)
249     :return: m_dot, v_e, p_e, Thrust, c_T, I_sp, T_e, M_e
250     '''
251 
252     Area_ratio = (ExitRadius/ThroatRadius)**2
253     # change np.float64 values to a float
254     gamma = float(gamma)
255     FEE_T = float(FEE_T)
256     R = float(R)
257     R = R*1000 # convert to J/kg-K
258 
259     M_e = float(MachSolve(gamma, Area_ratio, 5.0)[0])
260     T_e = FEE_T*((1+((gamma-1)/2)*M_e**2)**(-1))
261     v_e = M_e * np.sqrt(gamma * R * T_e)
262     p_e = p_starting * (1 + ((gamma - 1) / 2) * M_e ** 2) **
    (-gamma / (gamma - 1))
263     ExitRadius = ExitRadius/39.37 # convert to meters
264     ThroatRadius = ThroatRadius/39.37 # convert to
265 
266 
267     m_dot = (p_starting * np.pi * (ThroatRadius)**2) / (np.
    sqrt(FEE_T)) * \
268         np.sqrt(gamma/R)*((2/(gamma+1))**((gamma+1)/(2*(gamma
```

```
    -1))))
269
270     A_e = np.pi * ExitRadius**2
271     A_t = np.pi * ThroatRadius**2
272
273     # Control-volume momentum balance: F = m_dot*v_e + (p_e -
        p_a)*A_e.
274     # Chamber pressure only enters through c_T's
        normalization (p_c*A_t), not the thrust itself.
275     Thrust = m_dot * v_e + (p_e - p_ambient) * A_e
276     c_T = Thrust / (p_starting * A_t)
277     I_sp = Thrust / (m_dot * 9.81)
278
279     return m_dot, v_e, p_e, Thrust, c_T, I_sp, T_e, M_e
280
281 def FCC(FEE_F):
282     '''
283     This is a function that returns the performance of the
        flourine combustion chamber
284     :return:
285     '''
286
287
288     # The chemical equation is H2 +F2-> 2HF
289     molfrac_h2_react = 1
290     molfrac_f2_react = 1
291     molfrac_hf_react = 0
292     molfrac_h2_prod = 0
293     molfrac_f2_prod = 0
294     molfrac_hf_prod = 2
295
296
297     molfrac_h2_total = molfrac_h2_react - molfrac_h2_prod
298     molfrac_f2_total = molfrac_f2_react - molfrac_f2_prod
299     molfrac_hf_total = molfrac_hf_react - molfrac_hf_prod
300     sigma_nu = molfrac_hf_total + molfrac_h2_total +
        molfrac_f2_total
301     # print('sigma_nu = ', sigma_nu)
302
303     # Gibbs free energy of formation of H2, F2, and HF
304     del_h_h2 = h2_tableH(FEE_F)
305     del_h_f2 = f2_tableH(FEE_F)
306     del_h_hf = hf_tableH(FEE_F)
307
308     s_h2 = h2_tableS(FEE_F)
```

```
    s_f2 = f2_tableS(FEE_F)
    s_hf = hf_tableS(FEE_F)

    # Gibbs
    del_g_h2 = del_h_h2 - FEE_F*s_h2
    del_g_f2 = del_h_f2 - FEE_F*s_f2
    del_g_hf = del_h_hf - FEE_F*s_hf

    del_g_total = -del_g_f2 - del_g_h2 + del_g_hf
    print('del_g_total = ', del_g_total)
```

A.3 Iterater.py

```
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.optimize import fsolve

# Thermo tables live next to this file, so the code runs from any working directory
TABLES = Path(__file__).resolve().parent / 'tables'


def h2_tableH(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')

    idx = (h2_table['T']-Temp).abs().idxmin()
    T_closest = h2_table['T'][idx]
    h_hat_close = h2_table['H*-H*_298'][idx]
    if Temp > T_closest:
        T_upper = h2_table['T'][idx+1]
        h_hat_upper = h2_table['H*-H*_298'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = h2_table['T'][idx-1]
        h_hat_lower = h2_table['H*-H*_298'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_closest)*(Temp-T_closest)
```

```
    h_hat = h_hat*4184

    return h_hat

def f2_tableH(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    f2_table = pd.read_csv(TABLES / 'Fluorine_F2_table.csv')

    idx = (f2_table['T']-Temp).abs().idxmin()
    T_closest = f2_table['T'][idx]
    h_hat_close = f2_table['H*-H*_298'][idx]
    if Temp > T_closest:
        T_upper = f2_table['T'][idx+1]
        h_hat_upper = f2_table['H*-H*_298'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = f2_table['T'][idx-1]
        h_hat_lower = f2_table['H*-H*_298'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_closest)*(Temp-T_closest)
    h_hat = h_hat*4184

    return h_hat

def o2_tableH(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    o2_table = pd.read_csv(TABLES / 'o2_table.csv')

    idx = (o2_table['T']-Temp).abs().idxmin()
    T_closest = o2_table['T'][idx]
    h_hat_close = o2_table['H*-H*_298'][idx]
    if Temp > T_closest:
        T_upper = o2_table['T'][idx+1]
        h_hat_upper = o2_table['H*-H*_298'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
```

```
    else:
        T_lower = o2_table['T'][idx-1]
        h_hat_lower = o2_table['H*-H*_298'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_closest)*(Temp-T_closest)
    h_hat = h_hat

    return h_hat

def hf_tableH(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    hf_table = pd.read_csv(TABLES / 'HydrogenFluoride_HF_table.csv')

    idx = (hf_table['T']-Temp).abs().idxmin()
    T_closest = hf_table['T'][idx]
    h_hat_close = hf_table['h_hat'][idx]
    if Temp > T_closest:
        T_upper = hf_table['T'][idx+1]
        h_hat_upper = hf_table['h_hat'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = hf_table['T'][idx-1]
        h_hat_lower = hf_table['h_hat'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_closest)*(Temp-T_closest)
    h_hat = h_hat*4184

    return h_hat

def h2o_table_f(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')

    idx = (h2o_table['T']-Temp).abs().idxmin()
    T_closest = h2o_table['T'][idx]
```

```
    h_hat_close = h2o_table['h_hat'][idx]
    if Temp > T_cloest:
        T_upper = h2o_table['T'][idx+1]
        h_hat_upper = h2o_table['h_hat'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = h2o_table['T'][idx-1]
        h_hat_lower = h2o_table['h_hat'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)

    return h_hat

def oh_table_f(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    oh_table = pd.read_csv(TABLES / 'OH_table.csv')

    idx = (oh_table['T']-Temp).abs().idxmin()
    T_cloest = oh_table['T'][idx]
    h_hat_close = oh_table['h_hat'][idx]
    if Temp > T_cloest:
        T_upper = oh_table['T'][idx+1]
        h_hat_upper = oh_table['h_hat'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = oh_table['T'][idx-1]
        h_hat_lower = oh_table['h_hat'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)

    return h_hat

def h_table_f(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    h_table = pd.read_csv(TABLES / 'H_table.csv')
```

```
147
148     idx = (h_table['T']-Temp).abs().idxmin()
149     T_cloest = h_table['T'][idx]
150     h_hat_close = h_table['h_hat'][idx]
151     if Temp > T_cloest:
152         T_upper = h_table['T'][idx+1]
153         h_hat_upper = h_table['h_hat'][idx+1]
154         h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
155     else:
156         T_lower = h_table['T'][idx-1]
157         h_hat_lower = h_table['h_hat'][idx-1]
158         h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
159
160     return h_hat
161
162 def h2o_tableT(h_hat):
163     '''
164     :param h_hat: enthalpy in J/mol also known as heat of
    formation
165     :return Temp: temperature in K
166     '''
167     h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')
168
169     idx = (h2o_table['h_f']-h_hat).abs().idxmin()
170     h_hat_cloest = h2o_table['h_f'][idx]
171     T_close = h2o_table['T'][idx]
172     if h_hat > h_hat_cloest:
173         h_hat_upper = h2o_table['h_f'][idx+1]
174         T_upper = h2o_table['T'][idx+1]
175         T = T_close + (T_upper-T_close)/(h_hat_upper-
    h_hat_cloest)*(h_hat-h_hat_cloest)
176     else:
177         h_hat_lower = h2o_table['h_f'][idx-1]
178         T_lower = h2o_table['T'][idx-1]
179         T = T_close + (T_lower-T_close)/(h_hat_lower-
    h_hat_cloest)*(h_hat-h_hat_cloest)
180
181     return T
182
183 def h2_tableT(h_hat):
184     '''
185     :param h_hat: enthalpy in J/mol also known as heat of
    formation
```

```
    :return Temp: temperature in K
    '''
    h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')
    h_hat = h_hat/4184
    idx = (h2_table['H*-H*_298']-h_hat).abs().idxmin()
    h_hat_cloest = h2_table['H*-H*_298'][idx]
    T_close = h2_table['T'][idx]
    if h_hat > h_hat_cloest:
        h_hat_upper = h2_table['H*-H*_298'][idx+1]
        T_upper = h2_table['T'][idx+1]
        T = T_close + (T_upper-T_close)/(h_hat_upper-
    h_hat_cloest)*(h_hat-h_hat_cloest)
    else:
        h_hat_lower = h2_table['H*-H*_298'][idx-1]
        T_lower = h2_table['T'][idx-1]
        T = T_close + (T_lower-T_close)/(h_hat_lower-
    h_hat_cloest)*(h_hat-h_hat_cloest)

    return T

def hf_tableT(h_hat):
    '''
    :param h_hat: enthalpy in J/mol also known as heat of
    formation
    :return Temp: temperature in K
    '''
    f2_table = pd.read_csv(TABLES / '
    HydrogenFluoride_HF_table.csv')
    h_hat = h_hat/4184
    idx = (f2_table['H*-H*_298']-h_hat).abs().idxmin()
    h_hat_cloest = f2_table['H*-H*_298'][idx]
    T_close = f2_table['T'][idx]
    if h_hat > h_hat_cloest:
        h_hat_upper = f2_table['H*-H*_298'][idx+1]
        T_upper = f2_table['T'][idx+1]
        T = T_close + (T_upper-T_close)/(h_hat_upper-
    h_hat_cloest)*(h_hat-h_hat_cloest)
    else:
        h_hat_lower = f2_table['H*-H*_298'][idx-1]
        T_lower = f2_table['T'][idx-1]
        T = T_close + (T_lower-T_close)/(h_hat_lower-
    h_hat_cloest)*(h_hat-h_hat_cloest)

    return T

```

```
225
226 def oh_table_g(Temp):
227     '''
228     :param Temp: temperature in K
229     :return g_hat: Gibbs free energy in J/mol
230     '''
231     oh_table = pd.read_csv(TABLES / 'OH_table.csv')
232
233     idx = (oh_table['T']-Temp).abs().idxmin()
234     T_cloest = oh_table['T'][idx]
235     g_hat_close = oh_table['g_hat'][idx]
236     if Temp > T_cloest:
237         T_upper = oh_table['T'][idx+1]
238         g_hat_upper = oh_table['g_hat'][idx+1]
239         g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
240     else:
241         T_lower = oh_table['T'][idx-1]
242         g_hat_lower = oh_table['g_hat'][idx-1]
243         g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
244
245     return g_hat
246
247 def h_table_g(Temp):
248     '''
249     :param Temp: temperature in K
250     :return g_hat: Gibbs free energy in J/mol
251     '''
252     h_table = pd.read_csv(TABLES / 'H_table.csv')
253
254     idx = (h_table['T']-Temp).abs().idxmin()
255     T_cloest = h_table['T'][idx]
256     g_hat_close = h_table['g_hat'][idx]
257     if Temp > T_cloest:
258         T_upper = h_table['T'][idx+1]
259         g_hat_upper = h_table['g_hat'][idx+1]
260         g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
261     else:
262         T_lower = h_table['T'][idx-1]
263         g_hat_lower = h_table['g_hat'][idx-1]
264         g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
265
```

```
266     return g_hat
267 
268 def h2o_table_g(Temp):
269     '''
270     :param Temp: temperature in K
271     :return g_hat: Gibbs free energy in J/mol
272     '''
273     h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')
274 
275     idx = (h2o_table['T']-Temp).abs().idxmin()
276     T_closest = h2o_table['T'][idx]
277     g_hat_close = h2o_table['g_hat'][idx]
278     if Temp > T_closest:
279         T_upper = h2o_table['T'][idx+1]
280         g_hat_upper = h2o_table['g_hat'][idx+1]
281         g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(
    T_upper-T_closest)*(Temp-T_closest)
282     else:
283         T_lower = h2o_table['T'][idx-1]
284         g_hat_lower = h2o_table['g_hat'][idx-1]
285         g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(
    T_lower-T_closest)*(Temp-T_closest)
286 
287     return g_hat
288 
289 def h2_table_g(Temp):
290     '''
291     :param Temp: temperature in K
292     :return g_hat: Gibbs free energy in J/mol
293     '''
294     h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')
295 
296     idx = (h2_table['T']-Temp).abs().idxmin()
297     T_closest = h2_table['T'][idx]
298     g_hat_close = h2_table['g_hat'][idx]
299     if Temp > T_closest:
300         T_upper = h2_table['T'][idx+1]
301         g_hat_upper = h2_table['g_hat'][idx+1]
302         g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(
    T_upper-T_closest)*(Temp-T_closest)
303     else:
304         T_lower = h2_table['T'][idx-1]
305         g_hat_lower = h2_table['g_hat'][idx-1]
306         g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(
    T_lower-T_closest)*(Temp-T_closest)
```

```
307
308     return g_hat
309
310 def h2o_table_cp(Temp):
311     '''
312     :param Temp: temperature in K
313     :return cp_hat: heat capacity in J/mol-K
314     '''
315     h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')
316
317     idx = (h2o_table['T']-Temp).abs().idxmin()
318     T_closest = h2o_table['T'][idx]
319     cp_hat_close = h2o_table['cp_hat'][idx]
320     if Temp > T_closest:
321         T_upper = h2o_table['T'][idx+1]
322         cp_hat_upper = h2o_table['cp_hat'][idx+1]
323         cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(
    T_upper-T_closest)*(Temp-T_closest)
324     else:
325         T_lower = h2o_table['T'][idx-1]
326         cp_hat_lower = h2o_table['cp_hat'][idx-1]
327         cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(
    T_lower-T_closest)*(Temp-T_closest)
328
329     return cp_hat
330
331 def h2_table_cp(Temp):
332     '''
333     :param Temp: temperature in K
334     :return cp_hat: heat capacity in J/mol-K
335     '''
336     h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')
337
338     idx = (h2_table['T']-Temp).abs().idxmin()
339     T_closest = h2_table['T'][idx]
340     cp_hat_close = h2_table['cp_hat'][idx]
341     if Temp > T_closest:
342         T_upper = h2_table['T'][idx+1]
343         cp_hat_upper = h2_table['cp_hat'][idx+1]
344         cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(
    T_upper-T_closest)*(Temp-T_closest)
345     else:
346         T_lower = h2_table['T'][idx-1]
347         cp_hat_lower = h2_table['cp_hat'][idx-1]
348         cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(
```

```
    T_lower-T_cloest)*(Temp-T_cloest)

    return cp_hat

def oh_table_cp(Temp):
    '''
    :param Temp: temperature in K
    :return cp_hat: heat capacity in J/mol-K
    '''
    oh_table = pd.read_csv(TABLES / 'OH_table.csv')

    idx = (oh_table['T']-Temp).abs().idxmin()
    T_cloest = oh_table['T'][idx]
    cp_hat_close = oh_table['cp_hat'][idx]
    if Temp > T_cloest:
        T_upper = oh_table['T'][idx+1]
        cp_hat_upper = oh_table['cp_hat'][idx+1]
        cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = oh_table['T'][idx-1]
        cp_hat_lower = oh_table['cp_hat'][idx-1]
        cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)

    return cp_hat

def h_table_cp(Temp):
    '''
    :param Temp: temperature in K
    :return cp_hat: heat capacity in J/mol-K
    '''
    h_table = pd.read_csv(TABLES / 'H_table.csv')

    idx = (h_table['T']-Temp).abs().idxmin()
    T_cloest = h_table['T'][idx]
    cp_hat_close = h_table['cp_hat'][idx]
    if Temp > T_cloest:
        T_upper = h_table['T'][idx+1]
        cp_hat_upper = h_table['cp_hat'][idx+1]
        cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = h_table['T'][idx-1]
        cp_hat_lower = h_table['cp_hat'][idx-1]
```

```
        cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(T_lower-T_closest)*(Temp-T_closest)

    return cp_hat

def MachSolve(gamma, r_A, guess):
    '''
    Solve for Mach number given gamma, r_A, and guess
    :param gamma: the gamma of the gas
    :param r_A: the ratio of the areas
    :param guess: the initial guess of the Mach number
    :return: Mach number
    '''

    # M = var('M')
    front_cont = ((gamma+1)/2)**(-(gamma+1)/(2*(gamma-1)))
    sec_cont = (gamma-1)/2
    func = lambda M: (((front_cont)*((1+sec_cont*M**2)**((gamma+1)/(2*(gamma-1)))))/M) - r_A
    sol = fsolve(func, guess)

    return sol

def h2_tableS(Temp):
    '''
    :param Temp: temperature in K
    :return S_hat: entropy in J/mol-K
    '''
    h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')

    idx = (h2_table['T']-Temp).abs().idxmin()
    T_closest = h2_table['T'][idx]
    S_hat_close = h2_table['S'][idx]
    if Temp > T_closest:
        T_upper = h2_table['T'][idx+1]
        S_hat_upper = h2_table['S'][idx+1]
        S_hat = S_hat_close + (S_hat_upper-S_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = h2_table['T'][idx-1]
        S_hat_lower = h2_table['S'][idx-1]
        S_hat = S_hat_close + (S_hat_lower-S_hat_close)/(T_lower-T_closest)*(Temp-T_closest)

    return S_hat
```

```
431
432 def hf_tableS(Temp):
433     '''
434     :param Temp: temperature in K
435     :return S_hat: entropy in J/mol-K
436     '''
437     hf_table = pd.read_csv(TABLES / '
    HydrogenFluoride_HF_table.csv')
438
439     idx = (hf_table['T']-Temp).abs().idxmin()
440     T_closest = hf_table['T'][idx]
441     S_hat_close = hf_table['S'][idx]
442     if Temp > T_closest:
443         T_upper = hf_table['T'][idx+1]
444         S_hat_upper = hf_table['S'][idx+1]
445         S_hat = S_hat_close + (S_hat_upper-S_hat_close)/(
    T_upper-T_closest)*(Temp-T_closest)
446     else:
447         T_lower = hf_table['T'][idx-1]
448         S_hat_lower = hf_table['S'][idx-1]
449         S_hat = S_hat_close + (S_hat_lower-S_hat_close)/(
    T_lower-T_closest)*(Temp-T_closest)
450
451     return S_hat
452
453 def f2_tableS(Temp):
454     '''
455     :param Temp: temperature in K
456     :return S_hat: entropy in J/mol-K
457     '''
458     f2_table = pd.read_csv(TABLES / 'Fluorine_F2_table.csv')
459
460     idx = (f2_table['T']-Temp).abs().idxmin()
461     T_closest = f2_table['T'][idx]
462     S_hat_close = f2_table['S'][idx]
463     if Temp > T_closest:
464         T_upper = f2_table['T'][idx+1]
465         S_hat_upper = f2_table['S'][idx+1]
466         S_hat = S_hat_close + (S_hat_upper-S_hat_close)/(
    T_upper-T_closest)*(Temp-T_closest)
467     else:
468         T_lower = f2_table['T'][idx-1]
469         S_hat_lower = f2_table['S'][idx-1]
470         S_hat = S_hat_close + (S_hat_lower-S_hat_close)/(
    T_lower-T_closest)*(Temp-T_closest)
```

```
471
472     return S_hat
473 # Me = MachSolve(1.237115, 77.5426, 5)
474 # print('Me = ', Me)
475
476 def table_interp(filename, column, Temp):
477     '''
478     Linear interpolation of one column of a thermo table in
    temperature
479     :param filename: table file name inside ./tables
480     :param column: column to interpolate
481     :param Temp: temperature in K (must lie inside the table)
482     :return: interpolated value
483     '''
484     table = pd.read_csv(TABLES / filename)
485     T_min, T_max = table['T'].min(), table['T'].max()
486     if not T_min <= Temp <= T_max:
487         raise ValueError(f'T = {Temp} K is outside {filename}
    ({T_min}-{T_max} K)')
488     return float(np.interp(Temp, table['T'], table[column]))
489
490
491 H_F_298 = {'H2O': -241845.0, 'H2': 0.0, 'H': 217977.0, 'OH':
    38985.0, 'O2': 0.0}  # J/mol, from the tables
492
493
494 def sensible_enthalpy(species, Temp):
495     '''
496     Sensible enthalpy h(T) - h(298 K) in J/mol, built from
    the tables:
497       H2, O2 : tabulated H - H_298 (H2 in kcal/mol)
498       H2O    : tabulated h_f column
499       H      : constant cp (monatomic ideal gas)
500       OH     : from its formation enthalpy, since dh_f(OH) =
    h_OH - (1/2)h_H2 - (1/2)h_O2
501     '''
502     if species == 'H2':
503         return table_interp('Hydrogen_H2_table.csv', 'H*-H*
    _298', Temp)*4184
504     if species == 'O2':
505         return table_interp('o2_table.csv', 'H*-H*_298', Temp
    )
506     if species == 'H2O':
507         return table_interp('H2O_table.csv', 'h_f', Temp)
508     if species == 'H':
```

```
        return table_interp('H_table.csv', 'cp_hat', Temp)*(Temp - 298)
    if species == 'OH':
        return (table_interp('OH_table.csv', 'h_hat', Temp) - H_F_298['OH']
                + 0.5*sensible_enthalpy('H2', Temp) + 0.5*sensible_enthalpy('O2', Temp))
    raise ValueError(f'no enthalpy data for {species}')


def total_enthalpy(species, Temp):
    '''
    Formation + sensible enthalpy in J/mol (elements H2, O2 are zero at 298 K, the tables' reference)
    '''
    return H_F_298[species] + sensible_enthalpy(species, Temp)
```